



Széchenyi István Katolikus Technikum és Gimnázium

Szoftverfejlesztő és -tesztelő projektfeladat

„REpont Plusz”

Készítették:

Kriston-Széles Flórián,

Krizsai Tamás,

Kanta Áron

Ózd, 2025

Tartalomjegyzék

Bevezetés	3
Felhasználói dokumentáció	4
Rendszerkövetelmény	4
Hardver	4
Szoftver	4
Webalkalmazás indítása	5
Fejlesztői dokumentáció	6
Alkalmazott fejlesztői és csoportmunka eszközök	6
Adatbázis	6
Frontend	6
Backend	6
Adatbázis.....	7
Táblák Bemutatása:	7
Kapcsolat ábra:	11
ER diagram:.....	11
Frontend.....	12
Backend.....	22
Tesztelés	29
Továbbfejlesztési lehetőségek	36
Irodalomjegyzék	40

Bevezetés

A környezetvédelem mostanában az egyik legfelkapottabb témakör, melyben a hulladékkezelés és az újrahasznosítás van a középpontban. A műanyag palackok visszaváltása az egyik legelterjedtebb módszere az újrahasznosításnak, mert ezzel segíti csökkenteni a környezeti terhelést, és lehetőséget biztosít a műanyag újrahasznosítására.

Az volt a feladatunk, hogy egy valós, releváns problémára megoldást kellett találni egy webalkalmazás készítésével. A mi csoportunk feladata a REpont automaták köré épül. Célunk az automaták tartalmának a nyilvántartása, a felhasználók jutalmazása a palackok visszaváltásáért, remélve, hogy még több lesz az újrahasznosításra került termékek száma. A felhasználó a regisztrációnál megadja az irányítószámát, ez alapján tudjuk neki megmutatni a tartózkodási helyéhez közel eső REpont automatákat. Az ötlet onnan jött, hogy jelenleg sehol se lehet megtalálni, hogy hol vannak REpont gépek utca pontosan a megadott településeken.

A fejlesztés során az volt a cél, hogy felhasználóbarát, mobilbarát, jól átlátható, de funkcionalitásában kompromisszummentes legyen az webalkalmazásunk. A projekt létrehozásával, és egyben befejezésével azt is el szeretnénk volna érni, hogy fenntarthatóságra épülő közösségi szemléletet alakítsunk ki, ahol a környezettudatosság és a technológia összekapcsolhatóságának a lehetőségét mutassuk be.

<https://github.com/kristonflorian/Csoportos-feladat>

Felhasználói dokumentáció

Rendszerkövetelmény

Hardver

Rendszerkövetelmény

	Ajánlott	Minimális
Processzor (CPU)	Intel Core i3-2100 3.1 GHz	Intel Pentium G630
Operációs rendszer (OP)	Windows 10	Windows 10
RAM	4 GB	2 GB
Tárhely	1 GB	1 GB
Internet	Széles sávú	Széles sávú
GPU	NVIDIA GeForce 605	NVIDIA GeForce 405

1. kép: Rendszerkövetelmény

Szoftver

Legújabb NodeJS, PHP Composer, XAMPP, és egy aktívan támogatott webböngésző

Webalkalmazás indítása

Először el kell indítani a XAMPP-ot, azon belül az Apache Web Servert, és a MySQL modulokat, utána egy parancssorban elnavigálni oda, ahová le lett töltve/kitömörítve a Laravel, és ott lefuttatni egy ``php artisan migrate --seed`` parancsot, amivel fel lesz töltve az adatbázis részlegesen (ha kérdezi, hogy létre akarjuk-e hozni, akkor engedélyezzük). Utána vissza kell menni XAMPP-ra és ott a MySQL modulnál rálépni az Admin gombra, ami elvisz a phpMyAdmin adatbázis kezelő webes felületre, utána a baloldali menüben megkeresni a 'repon't' nevezetű adatbázist, és rákattintani utána felül az importálás gombra, és ott a fájl importálásnál betenni a 'settlementNmachineImport.sql' nevezetű fájlt, miután feltöltődött, utána letekerni és rákattintani az importálás gombra. Ezután az előbb említett parancssorban, hogy fusson a backend, ezt a parancsot kell beírni 'php artisan serve'. Utána kell nyitni egy másik parancssort úgy, hogy ami most lett elindítva azt nem zárjuk be, és az új parancssorral pedig elnavigálunk az Angular mappájába, ahova le lett töltve, és ott lefuttatunk egy 'ng serve' parancsot, és ezzel így már használható lesz a webalkalmazás.

Fejlesztői dokumentáció

Alkalmazott fejlesztői és csoportmunka eszközök

Git/GitHub: Project tárolása és verziómenedzselés

Discord: Kommunikáció

Trello: Feladatok elosztása a csapattagok között

Adatbázis

Visual Studio Code: SQL kód megírása

XAMPP: SQL kód tesztelése, Apache webszerver futtatása

MySQL: Itt van tárolva maga az adatbázis

Frontend

Visual Studio Code: A weboldal kódjának megírása

Angular: Keretrendszer a weboldalhoz

Figma: A weboldal megtervezése

Backend

Visual Studio Code: Laravel api, kontrollerek megírása és tesztelése

XAMPP: Adatbázis lokális futtatása és kezelése

Laravel: Keretrendszer az apihoz

Adatbázis

- a. Az adatbázis azzal a céllal lett létrehozva, hogy a REpont gépeknek a nyomon követésében, és a végfelhasználóknak ezeknek a gépeknek a hely alapú mutatásában segítsen. Könnyen meg lehet tudni egy gép jelenlegi adatait, például melyik településen van, a pontos címét, illetve milyen a megtelítettségének a szintje. Azon kívül, van még egy másik tábla, melyben különböző visszaváltott palackok száma díjazva vannak ajándékokkal (achievement-ek), amelyeket akkor lehet kapni, ha megadott mennyiségű üveg beváltásáért ajándékokkal jutalmazták meg a felhasználót, pl. kuponok különböző boltoknál. Van egy tábla, melyben Magyarország települései és azoknak az irányítószámai vannak eltárolva, ez a többi adattábla könnyebb összeköttetésére szolgál. Végül pedig a felhasználók is el vannak tárolva az adatbázisban, felhasználónév, jelszó, a felhasználó címe, hogy tudjuk melyik gépek vannak hozzá a legközelebb.

Táblák bemutatása:

1. Települések tábla

Mező	Rövid Leírás	Adattípus	Kulcs típusa	Attribútumok
zipcode	Itt a települések irányítószáma van tárolva	int	PRIMARY KEY	NOT NULL, UNIQUE
settlementname	A települések neve	varchar (75)	X	NOT NULL

2. Gépek tábla

Mező	Rövid Leírás	Adattípus	Kulcs típusa	Attribútumok
gid	Itt a gép azonosító száma van tárolva	int	PRIMARY KEY	NOT NULL, AUTO_INCREMENT
zipcode	A település irányítószáma, ahol a gép van	int	FOREIGN KEY	NOT NULL
fillpercentage	A gépnek a töltöttségi százaléka	int	X	X
machineaddress	A gépnek a pontos címe	varchar (100)	X	NOT NULL

3. Felhasználók tábla

Mező	Rövid Leírás	Adattípus	Kulcs típusa	Attribútumok
fid	Itt a felhasználó azonosító száma van tárolva	int	PRIMARY KEY	NOT NULL, AUTO_INCREMENT
username	A felhasználó felhasználóneve	varchar (30)	X	NOT NULL, UNIQUE
pass	A felhasználónak a jelszója van itt eltárolva	varchar (125)	X	NOT NULL
useraddress	A felhasználó pontos címe	varchar (100)	X	X
zipcode	A felhasználó irányítószáma	int	FOREIGN KEY	NOT NULL
bottlecount	Az újrahasznosított üvegek száma	int	X	NOT NULL
achievements	A legutóbb elért achievement	int	FOREIGN KEY	NOT NULL
email	A felhasználó e-mail címe	varchar (65)	X	NOT NULL UNIQUE

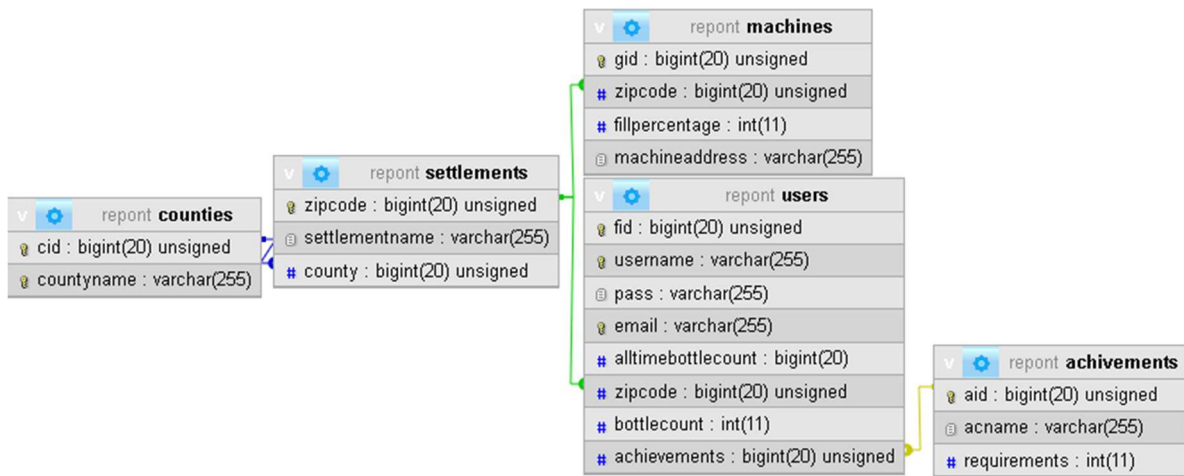
4. Megyék tábla

Mező	Rövid Leírás	Adattípus	Kulcs típusa	Attribútumok
cid	A megye azonosítója van itt	int	PRIMARY KEY	NOT NULL, AUTO_INCREMENT
countyname	A megye neve	varchar (30)	X	UNIQUE, NOT NULL

5. Achievementek tábla

Mező	Rövid Leírás	Adattípus	Kulcs típusa	Attribútumok
aid	Az achievement azonosítója van itt	int	PRIMARY KEY	NOT NULL, AUTO_INCREMENT
acname	Az achievement neve	varchar (75)	X	X
requirements	A szükséges üvegszám az eléréshez	int	X	NOT NULL

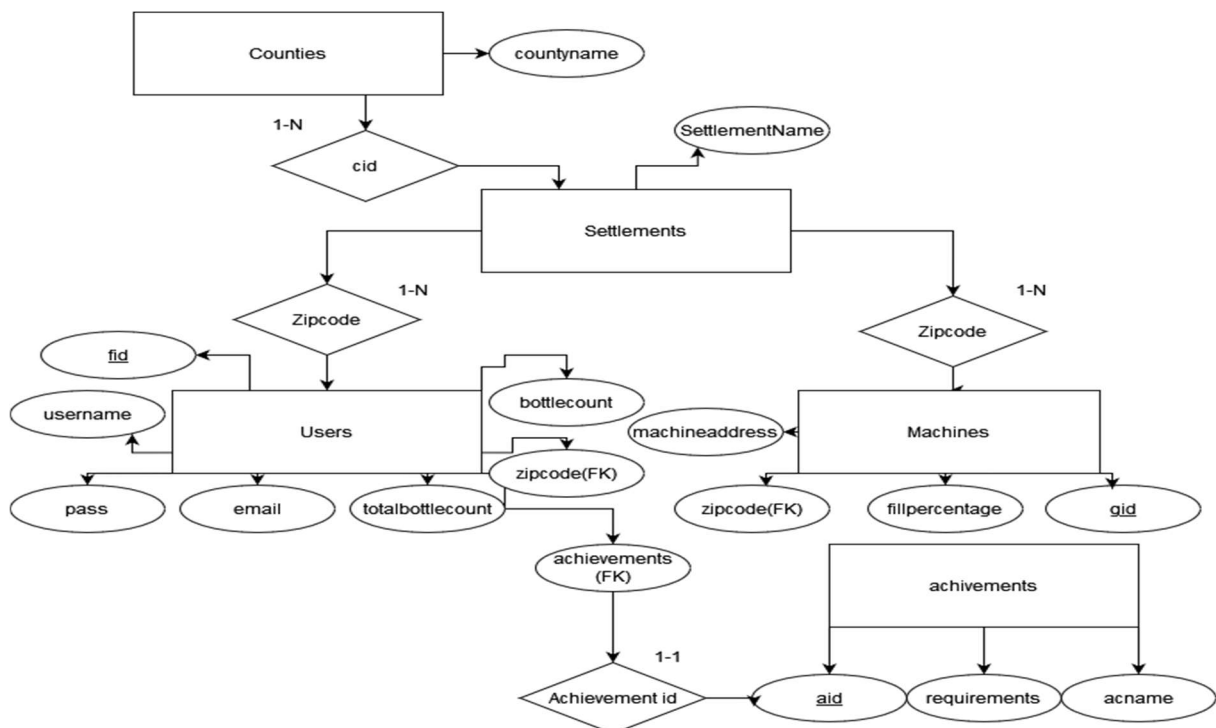
Kapcsolat ábra:



2. kép: Adatbázis kapcsolat ábra

Ahogy a képen látható, a megyék azonosítójával a települések össze vannak kötve, ez a keresőnek a használatához szükséges, mert így tudjuk kikeresni, hogy melyik megyéhez mely települések tartoznak. Ezen kívül a településeknek az irányítószáma is össze van kötve a felhasználók és a REpont gépek adattáblákkal, hogy tudjuk a felhasználó melyik településen lakik, a gépeknél pedig a keresőhöz kell, hogy meg tudjuk mutatni, hogy hol találhatóak.

ER diagram:



3. kép: Adatbázis ER diagram

Az ER diagram az adatbázist ábrázolja, és könnyedén láthatóak rajta a táblák közötti kapcsolatok.

Frontend

A weboldalunk stílusát a REpont|MOHU ihlette. Az oldal kinézetének újraalkotásában nagy hangsúlyt kapott a modernség mellett a jobb felhasználói élmény is. Több helyen 3D stílusban jelennek meg a gombok, információk, egyéb felhasználói elemek.



4. kép: Visszatérítések módjai, kártyák



5. kép: Visszatérítések módjai, kártyák, ha fölé állunk az egérrel

Az első képen az alaphelyzetben látható a konténer. Miután fölévittük a kurzort, és rámentünk egy választott kártyára az is hasonlóan fog viselkedni, mint a konténer. A bejelentkezési, és regisztrációs fülön is megjelennek a 3D-s animációk.

Bejelentkezés

Kedves Látogató! Jelentkezz be, hogy igénybe tudd venni egyéb szolgáltatásainkat!

Felhasználó név*

E-mail Cím*

Jelszó* 

Jelszó mégegyszer* 

Bejelentkezés

Regisztráció

6. kép: Bejelentkezés fül

A felemelkedő konténer, és háttérszín átváltozása itt is fellelhető. A Regisztráció gomb alaphelyzetben látható amikor betöltődik a fül. A Bejelentkezés gombon pedig az látható amikor a kurzort rávisszük a gombra, hogy ilyenkor ez is felveszi a 3D-s hatást, és átszíneződik. Ugyanez a formázás látható a regisztrációs fülön is.



Bejelentkezés

Kedves Látogató! Jelentkezz be, hogy igénybe tudj venni egyéb szolgáltatásainkat!

Felhasználó név*

E-mail Cím*

Jelszó* 

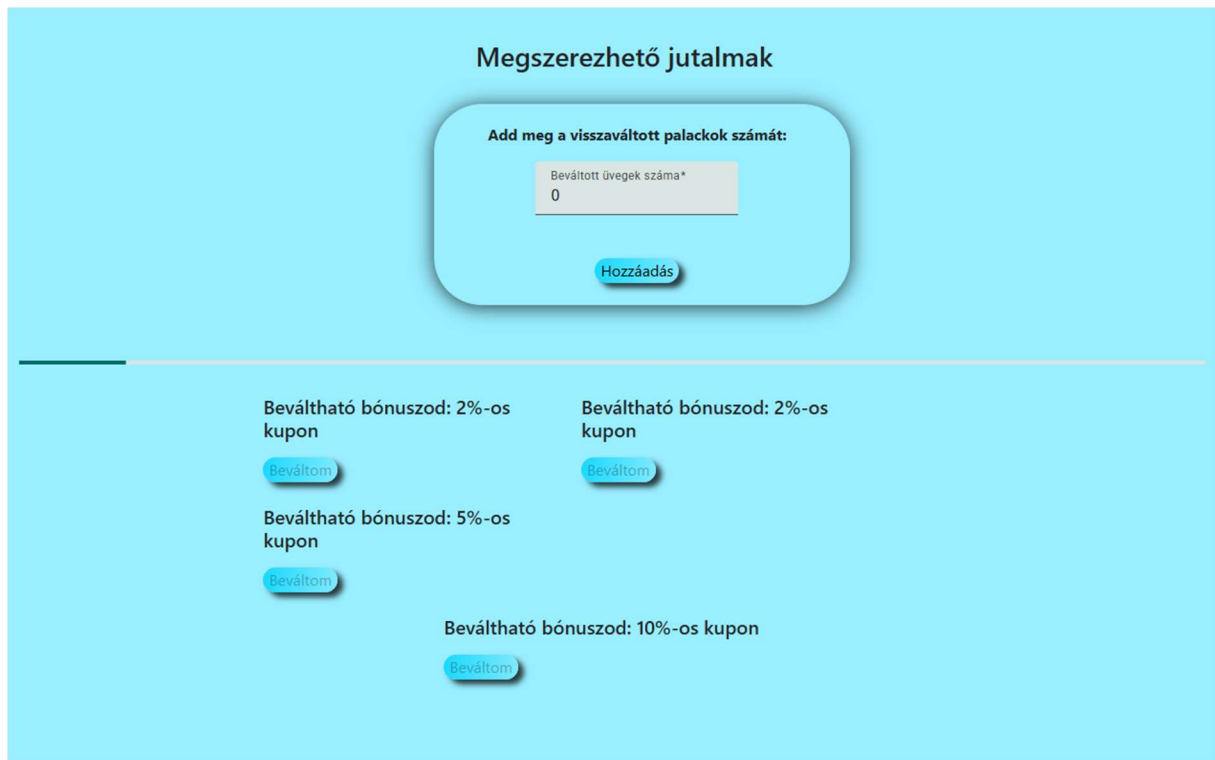
Jelszó megegyezés* 

Bejelentkezés

Regisztráció

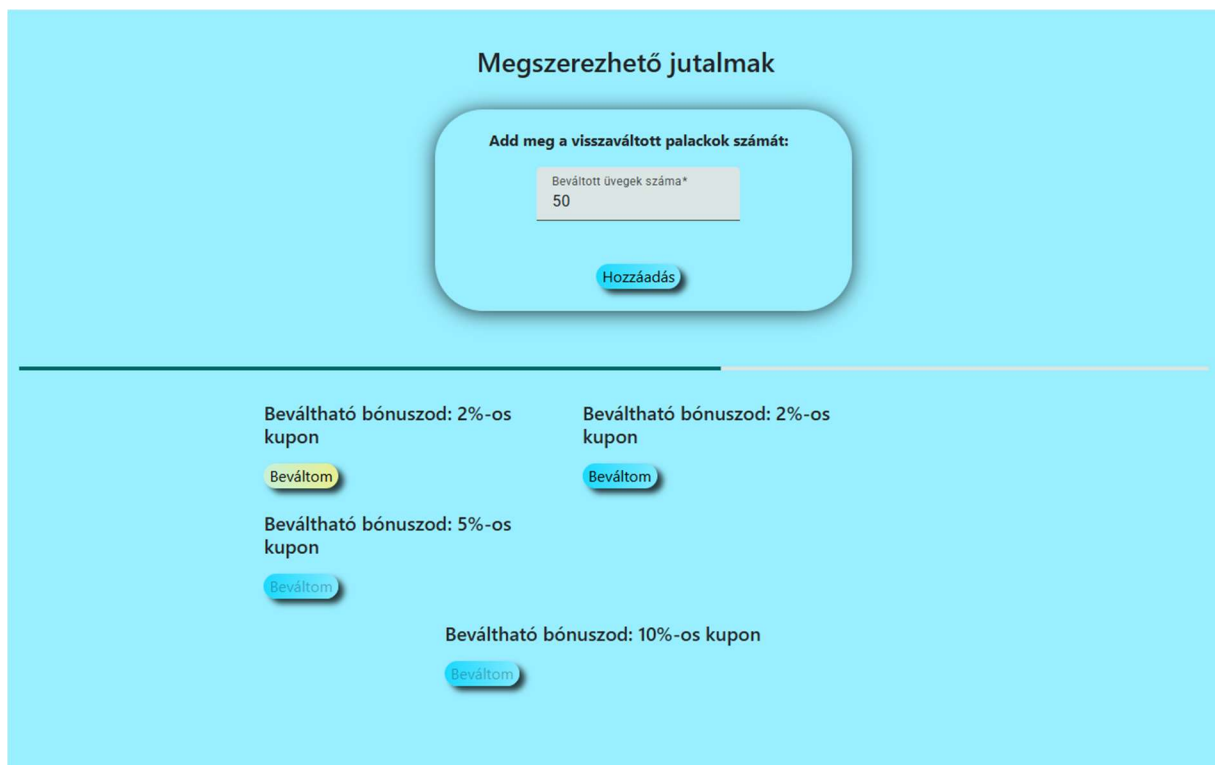
7. kép: Bejelentkezés fül, ha fölé állunk az egérrel

A regisztráció és bejelentkezés elvégzése után megnyílik előttünk az Achievementek fül, amin belül meg tudjuk adni mennyi palackot, vagy éppen üveget váltottunk vissza. Minél többet váltunk vissza, az annál kifizetődőbb, ugyanis összesen 4db utalvány is ütheti markunkat. Azaz 2db 2%-os, 1db 5%-os és 1db 10%-os kupon kerül a birtokunkba, ha szorgalmasan gyűjtögettünk.



8. kép: Achievementek föl 10 visszaváltott palack után

Megadott palack visszaváltása után a felhasználó jutalmakban részesül. Az első 25db visszaváltható üveg után kapjuk, egy 2%-os kupont. Ezután 50db-nál kapunk ebből még egyet, 75db-nál egy 5%-os, végül 100db után egy 10%-os kedvezménykupont kap a felhasználó.



9. kép: Achievementek 60 palack visszaváltása után

Ezen a részen is előbukkan a 3D-s animáció ugyanis mind a gombon, mind a konténerben láthatóak az előző formázások.



10. kép: Visszaváltott palackok számának megadása

A térképes kereső arra lett tervezve, hogy a felhasználó könnyen meg tudja találni az adott települést, amit keres, és átláthatóan kapjon információt.



11. kép: Vármegyék listázása a keresőben

A bal oldalt látható a side-bar menü, amiben a vármegyék találhatóak, a formázáson látható, hogy scroll-bar menüvel lett megvalósítva. Ha rákattintunk az egyik vármegyére akkor az adott elemen reaktív színváltozás észlelhető, és megjelenik az adott vármegyében található összes város.



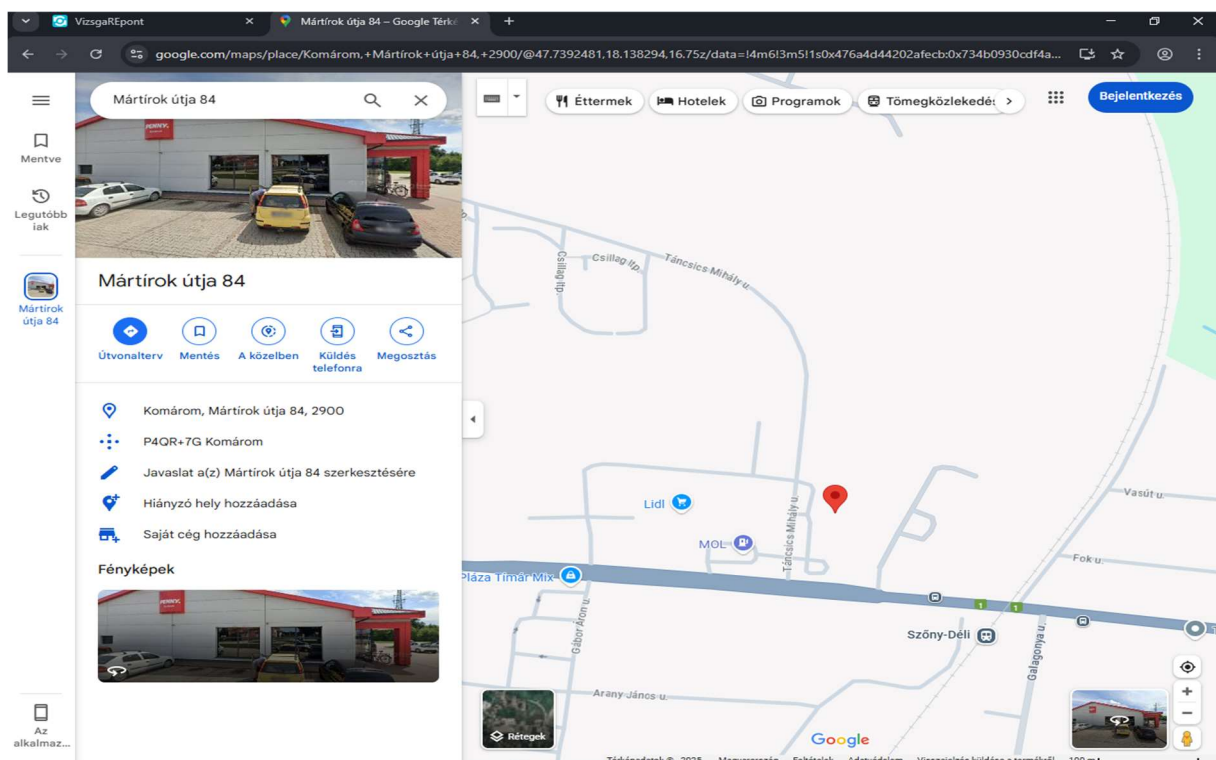
12. kép: Vármegye kereső városok listázása

Ha továbblépünk, és rákattintunk az adott városra, akkor megjelennek a kiválasztott városrészen fellelhető összes visszaváltási helyek, és a töltöttségi szintjeik.



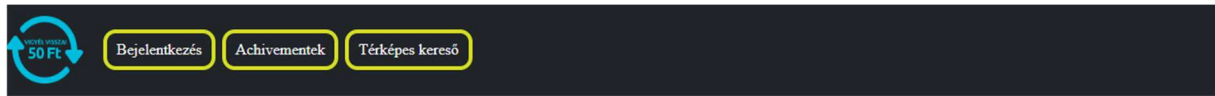
13. kép: REpont gépek listázása a keresőben

A címek linkké lettek átalakítva, amelyek átirányítják a felhasználót a Google Maps-ra, amely a pontos helyre navigál minket. A dizájn itt már jobban hasonlít egy táblázatra.



14. kép: REpont gép megmutatása Google Térképen

A navbar menüben megtalálható az ikonikus jel, ami a palackokon, és flakonokon is fellelhető. Ez a kép egy linkké is át lett konvertálva, ami a főoldalra navigál át minket, ha rányomunk.



15. kép: Navbar alaphelyzetben

A bejelentkezést, vagy regisztrációt követően megnyílik előttünk az Achievementek rész. Amíg a felhasználó nincs bejelentkezve, addig ez a gomb nem használható.

Amint bejelentkeztünk megnyílik előttünk az Achievementek szekció, a Bejelentkezés eltűnik, helyette a Kijelentkezés gomb jelenik meg.



16. kép: Navbar bejelentkezett felhasználó esetén

A footerben megjelennek a weboldalt készítőik nevei.



17. kép: Footer

Az oldal működése router outlettel lett megvalósítva, és az Achievementek nem hozzáférhető bejelentkezés nélkül, mert AuthGuard-al van védve azért, hogy aki nem felhasználó ne tudja használni.



18. kép: Az összes Router útvonal

A navbar és a footer külön Komponensek, és azok csak az app.component.html-ben vannak meghívva, nem pedig külön minden alelemben.

```
<app-header></app-header>
<router-outlet></router-outlet>
<app-footer></app-footer>
```

19. kép: app.component.html tartalma

A backend-el való kommunikáció néhány metódussal egy szolgáltatásban van megoldva, és ezek csak különböző komponensekben vannak meghívva a constructorban.

```
getCounties(){
  return this.http.get<County[]>('http://localhost:8000/api/county');
}
getSettlements(){
  return this.http.get<Settlement[]>('http://localhost:8000/api/settlement/search/county?countyname='+this.transport.getCounty());
}
getSettlementByZip(){
  return this.http.get<Settlement>('http://localhost:8000/api/settlement/'+this.transport.getZipcode());
}
getMachines(){
  return this.http.get<Machine[]>('http://localhost:8000/api/machine/search/'+this.transport.getZipcode());
}
```

20. kép: Az összes adat lekérés metódus

A fenti képen a különböző lekérések láthatóak, amik használva vannak a keresőhöz, mert a felhasználókezelés az egy másik szolgáltatásban történik. Az első a megyéket kéri le, a második a településeket azokban a megyékben, a harmadik a település adatokat kéri le irányítószám alapján, az utolsó pedig a gépeket hívja le irányítószám alapján.

```
CountyStorage(incomingData:string):void{
  this.localStorageTypes.forEach(item => {
    localStorage.removeItem(item);
  });
  localStorage.setItem(this.localStorageTypes[0],incomingData);
  this.CountyData.set(localStorage.getItem(this.localStorageTypes[0]));
}
ZipcodeStorage(incomingData:number):void{
  for (let i = 1; i < this.localStorageTypes.length; i++) {
    localStorage.removeItem(this.localStorageTypes[i]);
  }
  localStorage.setItem(this.localStorageTypes[1],incomingData.toString());
  const newStoredZip = localStorage.getItem(this.localStorageTypes[1])
  if (newStoredZip) {
    this.CityData.set(parseInt(newStoredZip))
  }
}
getCounty(){
  return this.CountyData.toString().slice(9,this.CountyData.toString().length-1);
}
getZipcode(){
  return this.CityData.toString().slice(9,this.CityData.toString().length-1);
}
ClearCountyAndZip(){
  this.CityData.set(null);
  this.CountyData.set(null);
  this.localStorageTypes.forEach(element => {
    localStorage.removeItem(element);
  });
}
```

21. kép: LocalStorageManagement szolgáltatás

A fenti képen a LocalStorageManagement service látható, melyben a Local Storage-ban tárolt megye, és település kezelése történik: felülírás, törlés, mentés és lekérés.

```
constructor(private auth: AuthService, private router: Router, private storage: LocalStorageManagement) {
  merge(this.email.statusChanges, this.email.valueChanges)
  .pipe(takeUntilDestroyed())
  .subscribe(() => this.updateErrorMessage())
}
ngOnInit(): void {
  this.storage.ClearCountyAndZip();
}
updateErrorMessage() {
  if (this.email.hasError('required')) {
    this.errorMessage.set('Kötelező email címet megadni')
  }
  else if (this.email.hasError('email')) {
    this.errorMessage.set('Nem helyes az email cím')
  }
}
registration() {
  this.auth.register(this.user)
  .subscribe({
    next: (response) => {
      this.router.navigateByUrl('/', {skipLocationChange: true}).then(() => {
        this.router.navigate(['/bejelentkezes'])
      })
    },
    error: (err) => {
      alert(err.message);
    }
  });
}
```

22. kép: Regisztrációs oldal TypeScript fájlja

Itt a regisztrációs oldal TypeScript kódrészlete van, itt benne van az email-cím ellenőrzés, és a regisztrációs metódus, ami a backendnek elküldve eltárolja a felhasználót az adatbázisban.

Backend

- a. Projektünk ezen része a Laravel keretrendszerre épülő REST alapú API-t foglalja magába. Ennek a célja, hogy a frontend számára adatokat biztosítson, és az onnan érkező kéréseket lekezelje. Ezen keresztül történik például a regisztráció, bejelentkezés, vagy a települések lekérdezése az adatbázisból.

```
12 Route::get('/settlement',[SettlementController::class,'index']);
13 Route::get('/settlement/{zip}',[SettlementController::class,'getByZipcode']);
14 Route::post('/settlement',[SettlementController::class,'store']);
15 Route::put('/settlement/{zip}',[SettlementController::class,'update']);
16 Route::get('/settlement/bigger/{zip}',[SettlementController::class,'biggerThanZip']);
17 Route::get('/settlement/smaller/{zip}',[SettlementController::class,'smallerThanZip']);
18 Route::get('/settlement/search',[SettlementController::class,'SearchByName']);
19
20 Route::get('/machine',[MachineController::class,'index']);
21 Route::get('/machine/{id}',[MachineController::class,'getById']);
22 Route::post('/machine',[MachineController::class,'store']);
23 Route::put('/machine/{id}',[MachineController::class,'update']);
24 Route::delete('/machine/{id}',[MachineController::class,'destroy']);
25 Route::get('/machine/morefull/{fill}',[MachineController::class,'moreFullThan']);
26 Route::get('/machine/emptier/{fill}',[MachineController::class,'emptierThan']);
27 Route::get('/machine/search',[MachineController::class,'SearchByAddress']);
28
29 Route::get('/user',[UserController::class,'index']);
30 Route::get('/user/{id}',[UserController::class,'getById']);
31 Route::put('/user/{id}',[UserController::class,'update']);
32 Route::post('/user',[UserController::class,'store']);
33 Route::delete('/user/{id}',[UserController::class,'destroy']);
34 Route::get('/user/more/{bottles}',[UserController::class,'moreBottleThan']);
35 Route::get('/user/less/{bottles}',[UserController::class,'lessBottleThan']);
36 Route::get('/user/search',[UserController::class,'SearchByName']);
37
38 Route::get('/achievement',[AchievementController::class,'index']);
39 Route::get('/achievement/{id}',[AchievementController::class,'getById']);
40 Route::post('/achievement',[AchievementController::class,'store']);
41 Route::put('/achievement/{id}',[AchievementController::class,'update']);
42 Route::delete('/achievement/{id}',[AchievementController::class,'destroy']);
43 Route::get('/achievement/bigger/{requirement}',[AchievementController::class,'biggerThanRequired']);
44 Route::get('/achievement/smaller/{requirement}',[AchievementController::class,'smallerThanRequired']);
45 Route::get('/achievement/search',[AchievementController::class,'SearchByRequired']);
```

23. kép: api.php útvonalak

Itt látható az api.php, amiben benne vannak a megírt végpontokhoz vezető elérési útvonalak. A frontend kódja erre hivatkozik és a felhasználó vagy adminisztrátor által elrendelt műveletet végrehajtja.

```

public function run(): void
{
    $names = ['Gyűjts össze 25 flakont!', 'Gyűjts össze 50 flakont!', 'Gyűjts össze 75 flakont!', 'Gyűjts össze 100 flakont!', 'Gyűjts össze 125 flakont!', 'Gyűjts össze 150 flakont!', 'Gyűjts össze 175 flakont!', 'Gyűjts össze 200 flakont!', 'Gyűjts össze 225 flakont!', 'Gyűjts össze 250 flakont!', 'Gyűjts össze 275 flakont!', 'Gyűjts össze 300 flakont!', 'Gyűjts össze 325 flakont!'];
    $requirements = [25, 50, 75, 100, 125, 150, 175, 200, 225, 250, 275, 300, 325];
    $length = count($names);
    for ($i=0; $i < $length; $i++) {
        Achievement::create(['acname'=> $names[$i], 'requirements'=> $requirements[$i]]);
    }

    $counties = ['Bács-Kiskun', 'Baranya', 'Békés', 'Borsod-Abaúj-Zemplén', 'Csongrád-Csanád', 'Fejér', 'Győr-Moson-Sopron', 'Hajdú-Bihar', 'Heves', 'Jász-Nagykun-Szolnok', 'Komárom-Esztergom', 'Nógrád', 'Pest', 'Somogy', 'Szabolcs-Szatmár-Bereg', 'Tolna', 'Vas', 'Veszprém', 'Zala'];
    $length2 = count($counties);
    for ($i=0; $i < $length2; $i++) {
        County::create(['countyname'=> $counties[$i]]);
    }
}

```

24. kép: databaseseeder.php használatával adatok feltöltése

Itt látható a databaseseeder, ennek a feladata alapvetően a keretrendszer, jelen esetben a Laravel által létrehozott adattáblák feltöltése adatokkal. Ez esetben az achievementek adattáblát tölti fel előre meghatározott értékekkel. A többi adattáblát külső sql fájlból importáljuk az adatbázisba, az adatok nagy mennyisége miatt.

```

public function index()
{
    return Settlement::all();
}

public function getByZipcode($zip)
{
    $sqt = Settlement::find($zip);
    if (is_null($sqt)) {
        return response()->json(["Error"=>"No settlement found with this zipcode"], 404);
    }
    else
    {
        return response()->json($sqt, 200);
    }
}

```

25. kép: Settlement index és getBy metódusok

Az index és a getById metódus (itt getByZipcode) feladata az adatok lekérése az adatbázisból és azok továbbítása JSON formátumban. E jelenlegi specifikus szituációban a településekhez tartozó Settlement Controller kódrészlete látható. Ennek segítségével hívható le az összes adat a települések adattáblából, vagy ha csak egy adott elemre van szükség, kivitelezhető az arra való keresés, táblától függ, hogy milyen adat alapján, ez lehet irányítószám, azonosító, cím stb. Utóbbi esetben a metódus kap egy paramétert, ebben az esetben egy irányítószámot, amire aztán rákeres az adatbázisban, ha megtalálja akkor leküldi az adott település adatait és egy 200-as státuszkódot, ellenkező esetben visszaad egy

hibaüzenetet, hogy nem található ilyen irányítószámú település, és ezt egy 404-es státuszkóddal is jelzi. Ez a metódus továbbítja a településeket a frontend számára, ahol megjelennek.

Lent látható a search metódus. Célja ugyanaz, mint a getByld eljárásnak, a különbség, hogy itt nem az elérési úton keresztül kell átadni a paramétert a metódus számára, hanem query paraméterben vagy JSON formátumban a request törzsén keresztül kapja meg. Itt sikeres kérés esetén csak az adott település továbbítódik, státuszkód nem, hiba esetén a response majdnem teljesen azonos, a hibaüzenet igazodik a metódus által keresett értékhez, itt a névhez, a státuszkód ebben az esetben is 404-es.

```
public function SearchByName(Request $request)
{
    $sqt=Settlement::where('settlementname','=', $request->settlementname);
    if($sqt->exists())
    {
        return $sqt->get();
    }
    else{
        return response()->json(['Error'=>'There is no settlement with this name'],404);
    }
}
```

26. kép: Settlement SearchBy metódus

```
if (!User::where('username','=', $request->username)->exists())
{
    $validate = Validator::make($request->all(),
    [
        "username"=>"required",
        "pass"=>"required",
        "email"=>"required",
        "zipcode"=>"required",
        "bottlecount"=>"required",
        "achievements"=>"required"
    ]);
    if ($validate->fails()) {
        return response()->json(["Error"=>"Missing required data"], 401);
    }
    $password = Hash::make($request->pass);
    $user = User::create(['username'=>$request->username, 'pass'=>$password, 'email'=>$request->email, 'zipcode'=>$request->zipcode,
    'bottlecount'=>$request->bottlecount, 'achievements'=>$request->achievements]);
    return response()->json(["New user added"=>$user->username], 201);
}
```

27. kép: User store metódus

A store metódus létfontosságú eleme az API működésének, különösen az User Controllerben. Enélkül a teljes rendszer működésképtelenné válna, mivel kivitelezhetetlen lenne a felhasználók adatainak eltárolása az adatbázisban. Ez az eljárás paraméterben megkapja request formájában egy felhasználó adatait. Ezután ezen adatok validálása következik, ahol a kód ellenőrzi, hogy minden kötelezően megadandó elemet megkapott-e a

metódus, ha igen akkor létrehoz egy új felhasználót, amit lement az adatbázisba, és küld egy választ JSON formátumban, 201-es státuszkóddal, amiben tudatja, hogy új felhasználó lett hozzáadva az adatbázishoz, és visszaküldi a felhasználónevet is. Hiba esetén 401-es státuszkódot továbbít, azzal az üzenettel, hogy valamelyik a kötelező adatok közül nem lett megadva. Ennek segítségével működik a frontendben a regisztráció és a bejelentkezés.

```
public function destroy($id)
{
    $user = User::find($id);
    if (is_null($user)) {
        return response()->json(["Error"=>"No user found with this id"], 404);
    }
    else
    {
        $user->delete();
        return response()->json("", 202);
    }
}
```

28. kép: User destroy metódus

A destroy metódus feladata az adott elem törlése az adatbázisból. A jelenlegi projekt esetében ezt alkalmazni kizárólag a felhasználóknál volt értelme, máshol ritkán fordul elő, hogy az adatok törlése indokolt legyen, illetve a legtöbb esetben hibákat is idézne elő, ha máshonnan adatokat törölnénk. Alapvető felépítése leginkább a GetById metódusra hasonlít, paraméterben kap egy id-t, ami alapján rákeres a felhasználóra az adatbázisban, ha nem találja meg 404-es státuszkóddal tér vissza, és egy hibaüzenettel miszerint nem található ilyen azonosítójú felhasználó. Ha sikeresen megtalálja a felhasználót, akkor törli azt az adatbázisból, és visszatér egy 202-es státuszkóddal, üzenet nélkül. Ennek segítségével tudja a felhasználó törölni a fiókját, ha valamilyen oknál fogva már nem szeretne többet belépni.

```

public function update(Request $request,$id)
{
    $user = User::find($id);
    if (is_null($user))
    {
        return response()->json(["Error"=>"No user found with this id"], 404);
    }
    $user->update($request->all());
    $user->save();
    return response()->json(['Message' => 'User successfully modified', 'username'=> $user->username], 200);
}

```

29. kép: User update metódus

Az update metódus felelős egy már meglévő adat megváltoztatásáért, ha például a felhasználó jelszót kíván változtatni, ez fog lefutni és kicserélni az adatbázisban az eltárolt jelszót. Működése hasonló a store metódushoz, de itt nem egy teljesen új elem kerül eltárolásra, hanem egy már meglévő módosul. Paraméterben kap egy requestet, amin keresztül az új adatok továbbítódnak, és egy id-t, ami a módosítandó elemet jelzi a kód számára. Először az kerül ellenőrzésre, hogy létezik-e felhasználó a megkapott azonosítóval, ha nem akkor 404-es státuszkódot ad vissza a metódus, azzal a státuszüzenettel, hogy nem található felhasználó ilyen id-vel. Ha viszont megtalálja a felhasználót, akkor a már meglévő adatok frissülnek és elmentődnek, majd egy státuszüzenettel visszatér a metódus, ami jelzi, hogy a felhasználó adatai módosultak, a felhasználónevet is visszaadja, illetve egy 200-as státuszkódot.

```

public function moreFullThan($fill)
{
    $containsmore = Machine::where('fillpercentage','>',$fill);
    if($containsmore->exists())
    {
        return $containsmore->get();
    }
    else
    {
        return response('There is no machine that is filled more than the input value!',404);
    }
}

public function emptierThan($fill)
{
    $containsless = Machine::where('fillpercentage','<',$fill);
    if($containsless->exists())
    {
        return $containsless->get();
    }
    else
    {
        return response('There is no machine that is filled less than the input value!',404);
    }
}

```

30. kép: Machine more/less metódusok

A moreFullThan és az emptierThan metódusok alapvető célja, hogy a kapott paramétert valamihez hasonlítsák, ez minden adattáblában más lehet, jelen esetben a REpont gépek töltöttségi szintjét vizsgálják, és ami oda tartozik azt visszaadják további felhasználásra.

Paraméterben megkapnak egy töltöttségi szinten, ezután kiszűrik azokat, amik abba a tartományba esnek. Ha nincs ilyen akkor 404-es státuszkódot adnak, és egy üzenetet, ami leírja, hogy nem található olyan gép, aminek a töltöttségi szintje több/kevesebb volt, mint a megkapott érték. Ha találhatóak adatok, akkor visszaadja azokat státuszkód és státuszüzenet nélkül.

```
public function FilterByCounty(Request $request)
{
    $county = County::where('countyname','=', $request->countyname)->first();
    $settlement=Settlement::where('county','=', $county->cid);
    if(!is_null($settlement))
    {
        return $settlement->get();
    }
    else{
        return response()->json(['Error'=>'There are no settlements in that county'],404);
    }
}
```

31. kép: County FilterBy metódus

A FilterByCounty metódus megkapja request formájában egy vármegyének a nevét, először ezt hozzá kell kötni az adott megyének az azonosítójához, mivel a települések adattáblában csak az id van eltárolva. Amint az összekapcsolás megtörtént, a where metódus, ami a Laravel keretrendszer része, rákeres az adott megyében lévő településekre. Ha talált ilyen településeket, akkor visszaadja azokat, ellenkező esetben JSON formátumban hibaüzenetet küld, 404-es státuszkóddal, és azzal az üzenettel, hogy a megadott megyében nem található település.

```

public function login(Request $request){
    $user = User::where('username','=',$request->username)->first();
    if ($user->exists()) {
        if ($request->email == $user->email) {
            if (Hash::check($request->pass, $user->pass)) {
                return $user;
            }
        }
    }
    else{
        return response()->json(["Error"=>"User doesn't exist with that username"], 401);
    }
}

```

32. kép: User login metódus

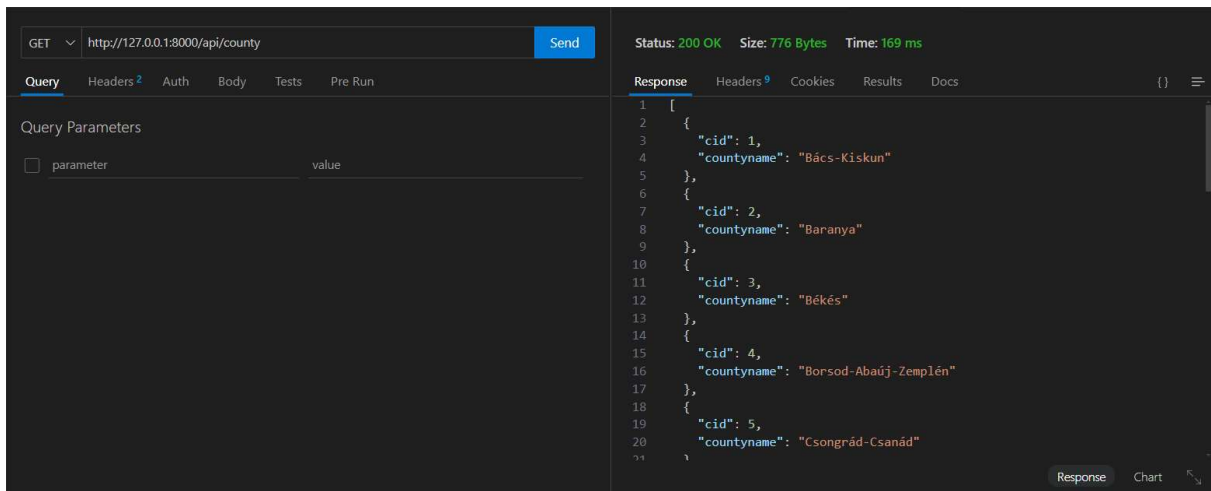
A login metódus request-ben megkapja egy felhasználó adatait. Ezután rákeres a megkapott felhasználónévre, és ha talál, akkor eltárolja a user változóba. Normális esetben egy felhasználónév csak egyszer szerepelhet az adatbázisban, de ha véletlenül valahogy belekerülne még egyszer, akkor az adatbázisban előrébb lévő felhasználó adataival fog dolgozni. A keresés után ellenőrzésre kerül, hogy a változó tartalmaz-e adatokat, ha nem az azt jelenti, hogy nem létező felhasználó adatai lettek megadva. Ha nem létezik a felhasználó az adatbázisban, akkor 401-es státuszkóddal és egy hibaüzenettel tér vissza a metódus. Ezután az kerül ellenőrzésre, hogy a felhasználó által beírt email cím megegyezik-e a felhasználónévhez tartozóval. Ha ez is sikeresen megtörténik, akkor jön a jelszó ellenőrzése. Ez a Laravel egyik beépített funkciójának, a Hash-nek a segítségével történik. Ez azért szükséges, hogy a felhasználó jelszavát ne lehessen kiolvasni egyenesen az adatbázisból. Ha a felhasználó által beírt jelszó megegyezik az adatbázisban eltárolt jelszóval, akkor a metódus visszaadja a felhasználó adatait, ez jelzi a frontendnek, hogy helyes és létező adatok lettek megadva, így a bejelentkezés sikeres.

Tesztelés

Az API tesztelése Thunder Client használatával történt, összesen 34 teszt segítségével. Lett ellenőrizve az összes response, sikeres lefutás és hiba esetén is.

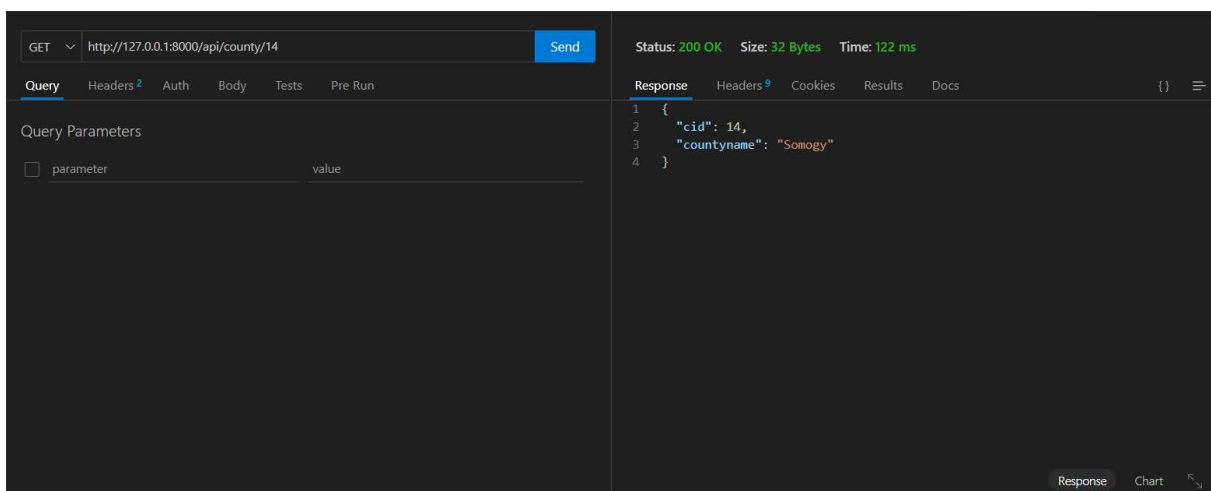
GET getmachine 2 months ago	GET settlementbiggerzip 2 months ago	
GET getsettlement 2 months ago	GET settlementsmallerzip 2 months ago	
GET getuser 2 months ago	GET settlementsearchbyname 2 months ago	
GET getachievement 2 months ago	PUT machineupdate 1 month ago	
GET getsettlementzip 2 months ago	DEL machinedelete 1 month ago	GET achievementsearch 2 months ago
GET getmachineid 1 month ago	GET machinemorefull 2 months ago	GET countyget 9 days ago
GET getachievementid 2 months ago	GET machineemptier 2 months ago	GET countygetbyid 3 mins ago
GET getuserid 6 days ago	GET machinesearch 2 months ago	GET countysearch 9 days ago
DEL userdelete just now	GET usermorebottle 2 months ago	
POST userpost just now	GET userlessbottle 2 months ago	
PUT userupdate 6 days ago	GET usersearch 2 months ago	
POST settlementpost 1 month ago	PUT achievementupdate 1 month ago	
POST machinepost 1 month ago	DEL achievementdelete 1 month ago	
POST achievementpost 1 month ago	GET achievementmore 2 months ago	
PUT settlementupdate 1 month ago	GET achievementsmaller 2 months ago	

33. kép: Az összes tesztet Thunder Client-ben



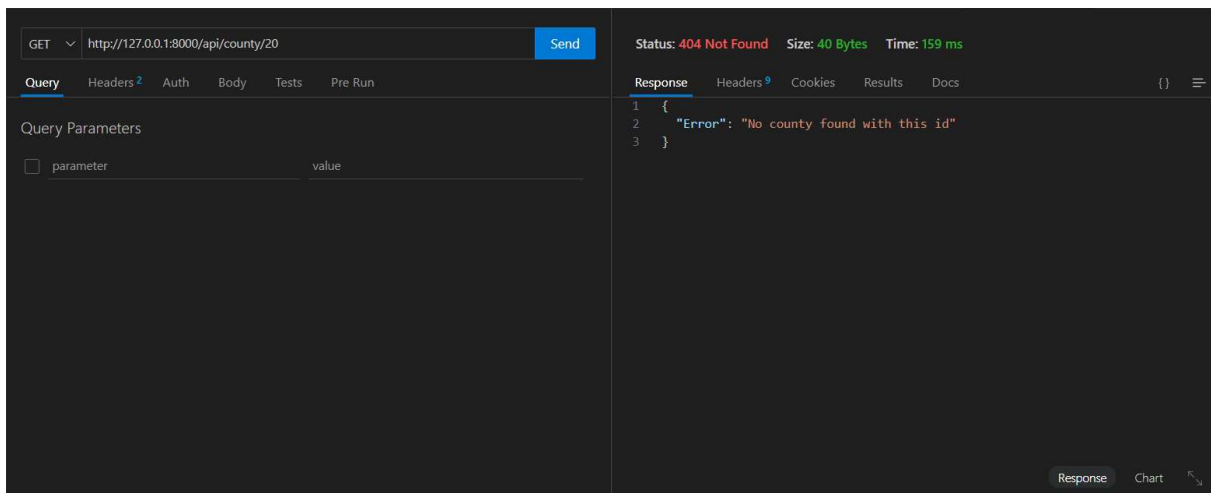
34. kép: Thunder Client county get metódus teszt

Ez a teszt a CountyController.php index metódusát teszteli, ami leküldi az összes megyét id-vel együtt. Sikeresen lefut a teszt, 200-as státuszkódot ad vissza.



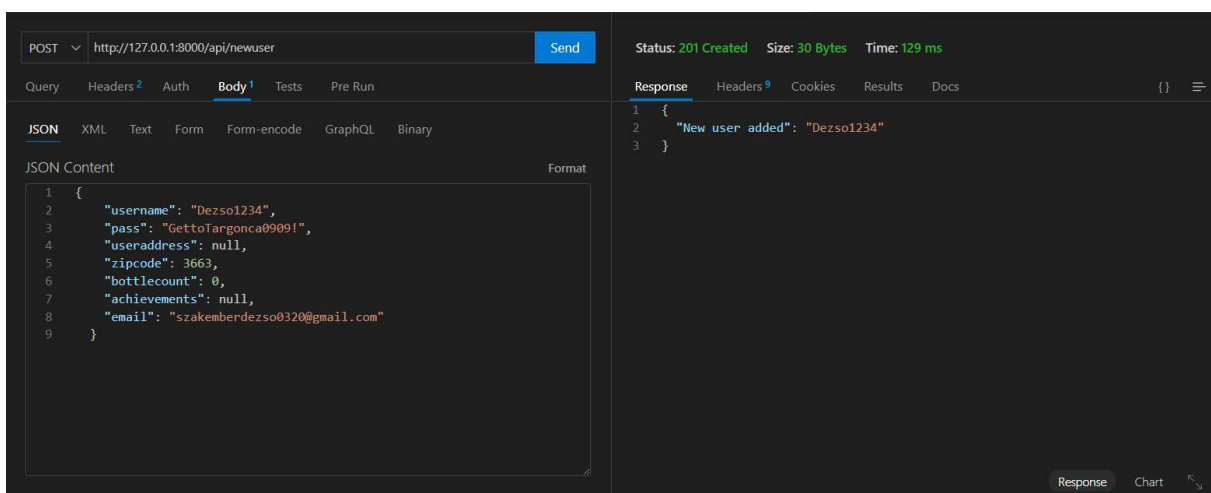
35. kép: Thunder Client county getByld metódus teszt

Ez a teszt a CountyController.php getByld metódusát teszteli, ami leküldi a megadott azonosítóval ellátott megyét. Sikeresen lefut a teszt, 200-as státuszkódot ad vissza.



36. kép: Thunder Client county getByld metóds hiba tesztelése

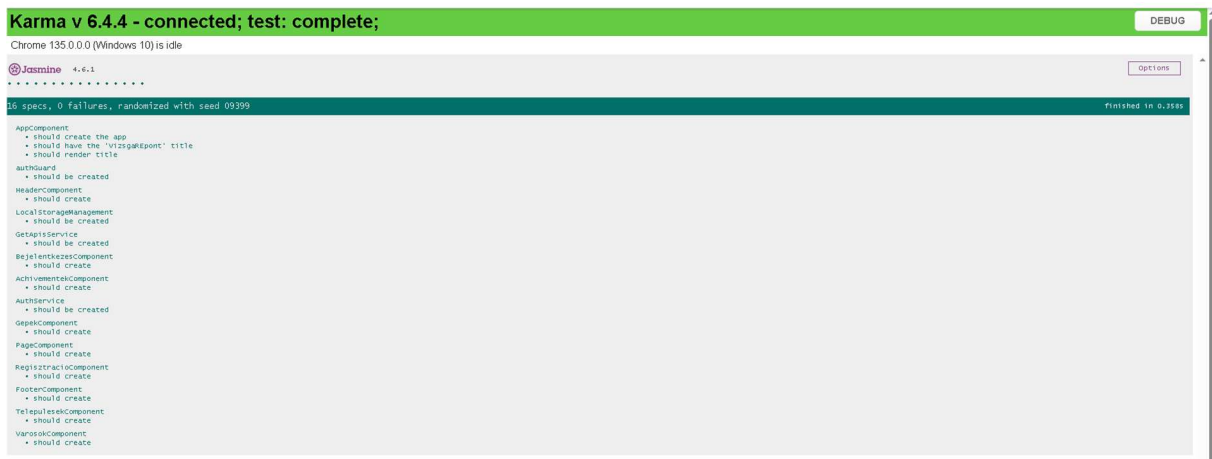
Ez a teszt a CountyController.php getByld metódusát teszteli, ami leküldi a megadott azonosítóval ellátott megyét. Ebben az esetben a teszt hibát jelez, mivel nincs ilyen azonosítójú megye az adatbázisban, és a kódban előre definiált 404-es hibakóddal és egy hibaüzenettel tér vissza, ami jelzi, hogy nem található vármegye ilyen id-vel.



37. kép: Thunder Client user post metóds teszt

Ez a teszt a UserController.php post metódusát teszteli, ami az egyik legfontosabb az oldal működése szempontjából. JSON formátumban kapja meg az adatbázis számára elküldendő adatokat. Mivel itt minden szükséges adat meg lett adva és helyes is, ezért a teszt sikeresen lefut és visszaad egy 201-es státuszkódot, jelezve, hogy új elem lett létrehozva az adatbázisban.

A frontend Jasmine-el lett tesztelve, és a frontendnek a problémás részei is külön lettek tesztelve manuálisan. Ez a teszt a különböző komponensek és service-ek létrehozását teszteli.



38. kép: Jasmine tesztelés

The image shows a registration form titled 'Regisztráció' (Registration) on a yellow background. The form includes fields for 'Felhasználó név*' (Username), 'Irányítószám*' (Postal code), 'E-mail cím*' (Email address), 'Jelszó*' (Password), and 'Jelszó mégegyszer*' (Repeat password). The 'E-mail cím*' field contains the text 'test' and has a red error message below it: 'Nem helyes az email cím' (Email address is not correct). A green 'Regisztrálás' (Registration) button is at the bottom.

39. kép: Nem helyes email cím megadása

Regisztrációnál minden mező ellenőrizve van, hogy ne legyen üres, és az email címnél az is, hogy egy érvényes email cím legyen, a jelszónál pedig az, hogy a minimum karakter szám az a 8.

Bejelentkezéskor ugyanazek az ellenőrzések megvannak, és működnek.

Bejelentkezés

Kedves Látogató! Jelentkezz be, hogy igénybe tudd venni egyéb szolgáltatásainkat!

Felhasználó név*

Rossz a felhasználónév!

E-mail Cím*

Kötelező email címet megadni

Jelszó*

Nem írtál be jelszót

Jelszó megegyezés*

Bejelentkezés

Regisztráció

40. kép: Minden adatmező rosszul van kitöltve

Add meg a visszaváltott palackok számát:

Beváltott üvegek száma*

-1

Összes üveged száma: 25

Ne adj meg negatív számot

Hozzáadás

41. kép: Negatív számot adunk meg

Ezen a képen az látható, hogy a felhasználó negatív számot ad meg, ezen kívül van még külön ellenőrzés arra, hogy a felhasználó egyáltalán számot ad-e meg, ami az alábbi ábrán lesz látható.

Add meg a visszaváltott palackok számát:

Beváltott üvegek száma*

test

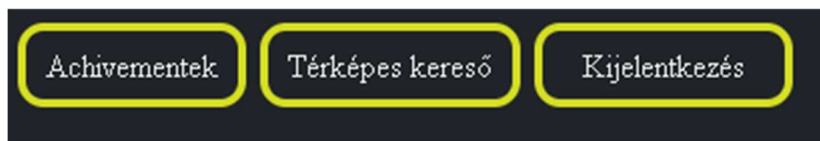
Összes üveged száma: 25

Nem számot adtál meg!

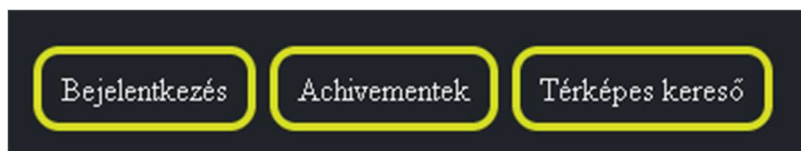
Hozzáadás

42. kép: Nem számot adunk meg

Minden oldalról el lehet érni a releváns szekciókat például, ha be vagyunk jelentkezve akkor a bejelentkezés fülhöz nincs hozzáférésünk, ha nem vagyunk bejelentkezve akkor a kijelentkezés gombot nem látjuk, és ha rákattintunk az achivementek gombra, akkor az is a bejelentkezés oldalra visz.



43. kép: Bejelentkezett felhasználó

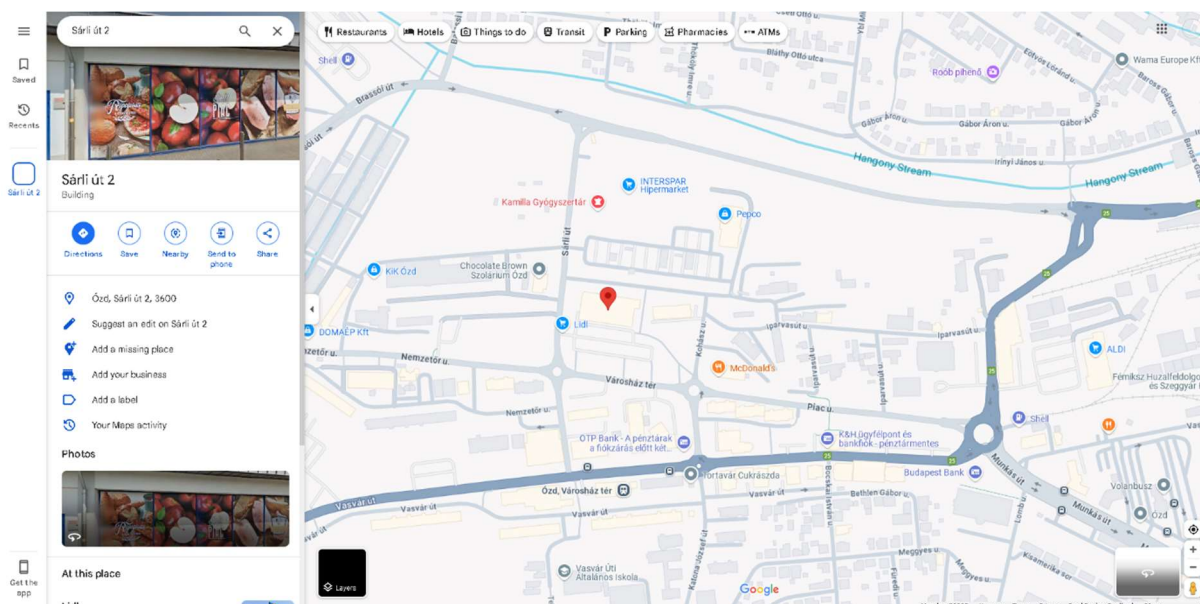


44. kép: A felhasználó nincs bejelentkezve

A Google Maps beágyazás is működik gond nélkül.



45. kép: Itt látható egy visszaváltó gép Ózdon.



46. kép: A Repont gép címére kattintva ide lesz átvetve a felhasználó

Az egész webalkalmazás az reszponzív, és mobil, tablet eszközökön is használható.



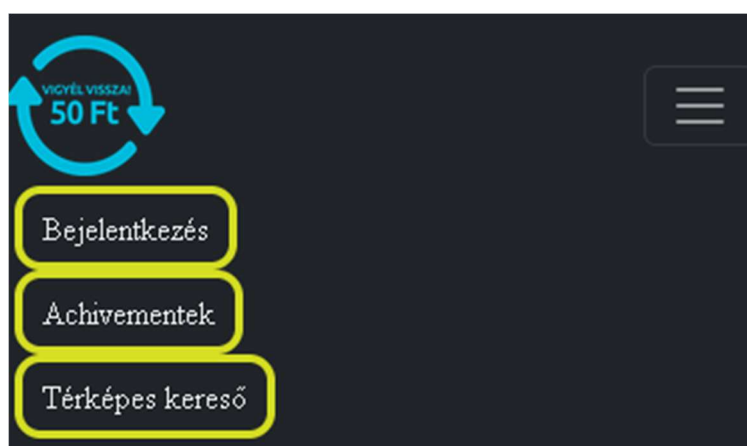
47. kép: Főoldal telefonos eszközön

A fenti ábrán a főoldal látható, ahogyan egy mobil eszközön kinéz, és az egész oldal tartalma megtekinthető telefonos eszközön.



48. kép: A főoldal alsó része

A navbar-nak is van rezponzivitása, és működik, gombbal lehet elrejteni és mutatni.



49. kép: Navbar mobilos eszközön

Vármegye kereső	Városok ebből a megyéből	Töltött ségi szint	cím
Bács-Kiskun	3400 (Mezőkövesd)		
Baranya	3413 (Cserépfalu)	86 %	Sárli út 2.
Békés	3432 (Emőd)	59 %	Sárli utca 4.
Borsod-Abaúj-Zemplén	3433 (Nyékládháza)	20 %	Brassói út 3.
Csongrád-Csanád	3434 (Mályi)		
Fejér	3441 (Mezőkeresztes)	22 %	Bolyki főút 67.
Győr-Moson-Sopron	3450 (Mezőcsát)	90 %	Bolyki Fő út 25.
Hajdú-Bihar	3458 (Tiszakeszi)	81 %	Mekcsey utca 153.
Heves	3503 (Miskolc)		
Jász-Nagykun-Szolnok	3516 (Miskolc)	24 %	Jácsi Oszkár utca 4.
Komárom-Esztergom	3521 (Miskolc)		

50. kép: A kereső kinézete mobil eszközön

A gépkeresőnek is meg van csinálva a rezponzivitása, és a különböző felbontásokkal és képarányokkal sincsen gond.

Vármegye kereső	Városok ebből a megyéből	Töltött ségi szint	cím
Bács-Kiskun	3400 (Mezőkövesd)		
Baranya	3413 (Cserépfalu)		
Békés	3432 (Emőd)	86 %	Sárli út 2.
Borsod-Abaúj-Zem	3433 (Nyékládháza)		
Csongrád-Csanád	3434 (Mályi)		
Fejér	3441 (Mezőkeresztes)	59 %	Sárli utca 4.
Győr-Moson-Sopron	3450 (Mezőcsát)		
Hajdú-Bihar	3458 (Tiszakeszi)		
Heves	3503 (Miskolc)		
Jász-Nagykun-Szol	3516 (Miskolc)	20 %	Brassói út 3.
Komárom-Esztergo	3521 (Miskolc)		
Nógrád	3524 (Miskolc)		
Pest	3525 (Miskolc)	22 %	Bolyki főút 67.
Somogy	3526 (Miskolc)		
Szabolcs-Szatmár-E	3527 (Miskolc)		
Tolna	3528 (Miskolc)	90 %	Bolyki Fő út 25.
Vas	3529 (Miskolc)		
Veszprém	3530 (Miskolc)		
	3531 (Miskolc)	81 %	Mekcsey utca 153.
	3532 (Miskolc)		

51. kép: A vármegye kereső tablet eszközön tekintve

Bejelentkezés

Kedves Látogató! Jelentkezz be, hogy igénybe tudd venni egyéb szolgáltatásainkat!

Felhasználó név*

E-mail Cím*

Jelszó*

Jelszó mégegyszer*

Bejelentkezés

Regisztráció

52. kép: A bejelentkezés oldal mobilon

Regisztráció

Kedves Látogatónk! A szolgáltatásunk igénybevételéhez, kérjük, regisztráljon az alábbi adatok megadásával!

Felhasználó név*

Irányítószám*
0

E-mail Cím*

Jelszó*

Jelszó mégegyszer*

Regisztrálás

53. kép: Regisztrációs menü telefonon

A bejelentkezés és a regisztráció is működik gond nélkül.

Megszerezhető jutalmak

Add meg a visszaváltott palackok számát:

Beváltott üvegek száma*
0

Hozzáadás

Beváltható bónuszod: 2%-os kupon	Beváltható bónuszod: 2%-os kupon
Beváltom	Beváltom
Beváltható bónuszod: 5%-os kupon	
Beváltom	
Beváltható bónuszod: 10%- os kupon	
Beváltom	

54. kép: Achievementek oldal reszponzív kinézete

A jutalmak oldal is akadálymentesen működik.

Továbbfejlesztési lehetőségek

Az alkalmazást úgy lehetne tovább fejleszteni, hogy a táblázat alapú gépkereső helyett egy térképes alapút lehetne használni, és a megszerezhető jutalmaknál nem a felhasználó adja meg a beváltott üvegek számát, hanem a gépek küldik el az adattáblákba.

Irodalomjegyzék

<https://repoint.hu/hu>

https://hu.wikipedia.org/wiki/Magyarország_vármegyéi

<https://material.angular.io/guides>

<https://laravel.com/docs/11.x/>

<https://cssbuttons.app/>

<https://getbootstrap.com/>